System Design: Cloud IoT Monitoring with Blockchain-Verified Service Reports

1. Tech Stack

Cloud platform: AWS, region ap-south-1 (Mumbai)

Device layer

- Simulator: Python 3.12 with AWS IoT Device SDK for Python v2 (`awsiotsdk`)
- Protocol and security: MQTT over TLS with one X.509 certificate per device

Cloud services

- Ingestion and routing: AWS IoT Core (Things, policies, Rules Engine)
- Fault handling: AWS Lambda (Python 3.12)
- Database: Amazon DynamoDB, provisioned mode, with TTL
- Alerts: Amazon SNS email
- File storage: Amazon S3, private bucket with SSE-S3 encryption
- Hosting: Amazon EC2 t3.micro, Ubuntu Server 24.04 LTS, systemd
- Secrets: AWS Systems Manager Parameter Store (standard tier, SecureString)
- Access control: AWS IAM roles with least privilege
- Monitoring: Amazon CloudWatch Logs
- Admin access: AWS Systems Manager Session Manager

Application layer

- Dashboard: Streamlit
- AWS access from Python: boto3
- Hashing: Python `hashlib` (SHA-256)

Blockchain layer

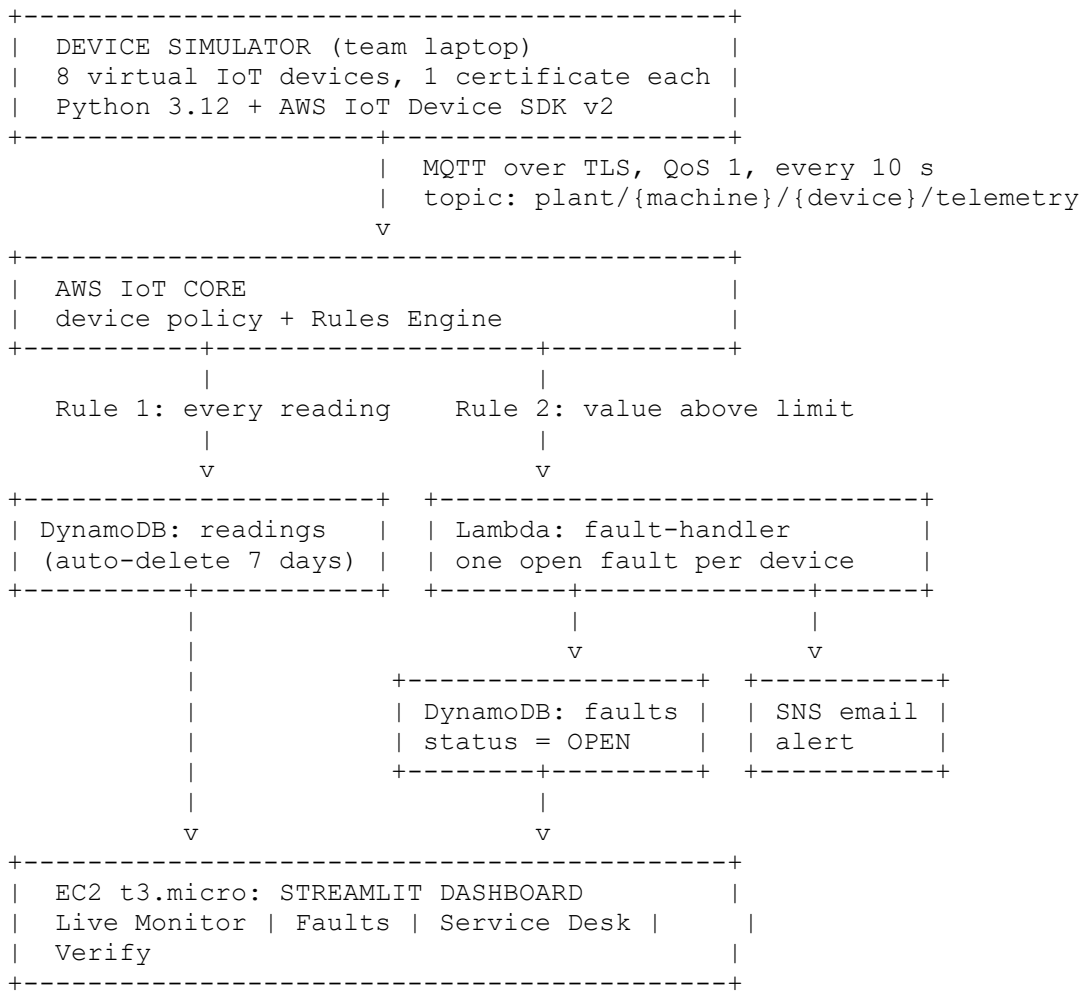
- Smart contract: Solidity 0.8.x
- Development and testing: Hardhat on Node.js 22 LTS
- Network: Ethereum Sepolia testnet
- Chain access from Python: web3.py with `eth_account` signing
- RPC provider: Alchemy Free plan
- Test ETH: pk910 Sepolia PoW faucet
- Block explorer: Etherscan Sepolia

Development tools: Git and GitHub, AWS CLI v2, `reportlab` (test PDFs)

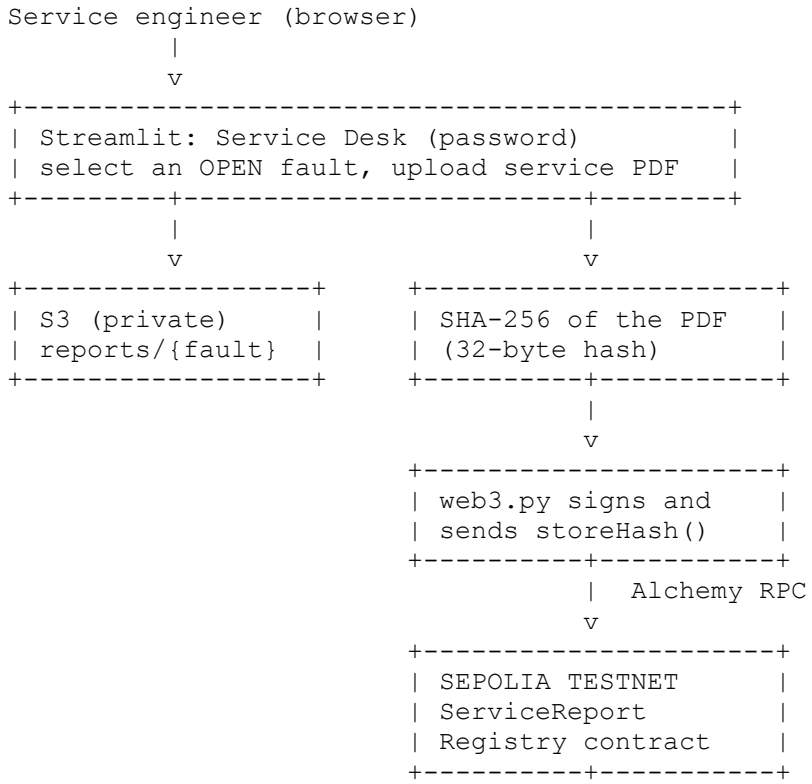
---

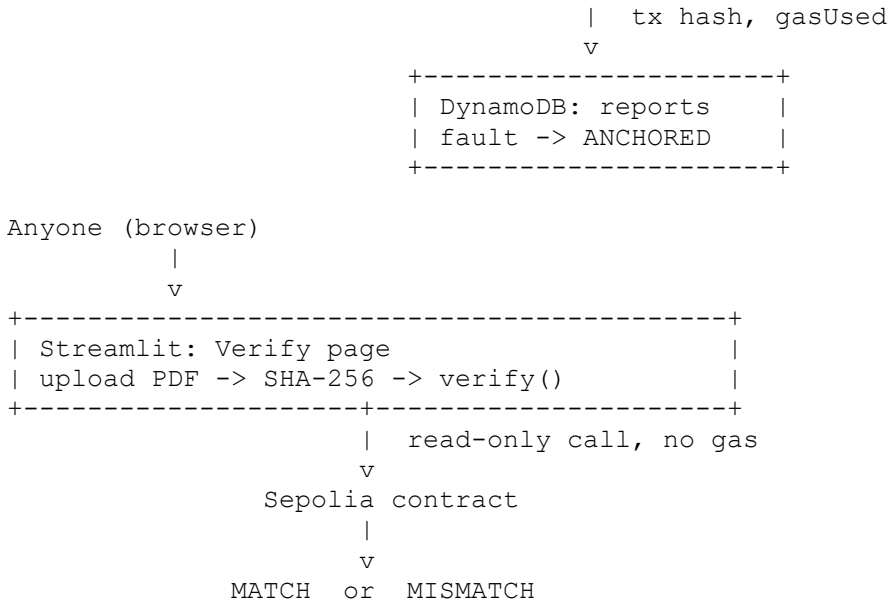
2. Architecture Design

Flowchart A: Monitoring path



Flowchart B: Service report and verification path





Supporting services (not shown in the charts): IAM roles control all access, SSM Parameter Store holds the wallet key, RPC URL and contract address, and CloudWatch Logs records Lambda and rule activity.

---

3. Technical Description

Device layer

- Eight simulated devices are spread over two machines. Each is a separate IoT Thing.
- **Machine A (Motor-Pump Set):**
  - A1-MOTOR-TEMP: motor winding temperature
  - A2-MOTOR-CURR: motor supply current
  - A3-MOTOR-VIB: motor drive-end bearing vibration
  - A4-PUMP-VIB: pump bearing vibration
- **Machine B (Conveyor Drive):**
  - B1-MOTOR-TEMP: conveyor motor casing temperature
  - B2-MOTOR-CURR: conveyor motor supply current
  - B3-GBOX-VIB: gearbox housing vibration
  - B4-GBOX-TEMP: gearbox oil temperature
- **Alert limits:** temperature above 90 °C, vibration above 7 mm/s, current above 12 A.
- **Simulation:** each device sends one JSON reading every 10 seconds, following a random walk inside its normal range. The console commands `inject <device_id>` and `clear <device_id>` raise or restore a value so faults can be shown on demand.
- **Message format:** `{device_id, machine_id, part, sensor_type, value, unit, sent_at}`.

Ingestion layer

- Each device connects with its own certificate. The policy lets a device connect only as its own Thing name and publish only to its own topic, so a device cannot publish as another device.
- Rule 1 writes every reading to the `readings` table and adds a timestamp and an expiry time.
- Rule 2 uses an SQL `WHERE` clause with the three limits and invokes Lambda only for readings above a limit.
- Both rules write to CloudWatch Logs when an action fails.

Fault handling

- The Lambda function creates a fault ID and runs one DynamoDB transaction that writes the fault item and an "active" marker for that device.
- The marker uses a "must not already exist" condition. If it already exists, an open fault is present, so no new fault or email is created.
- A new fault sends one SNS email with device, part, value, limit, time and fault ID.
- A fault stays OPEN until its service report is anchored on the blockchain.

Data model

- `readings`: key is device ID plus timestamp. It holds machine, part, sensor type, value, unit and an expiry time. Readings are deleted automatically after 7 days.
- `faults`: key is device ID plus fault ID. It holds part, value, limit, status (OPEN or ANCHORED) and creation time.
- `reports`: key is fault ID. It holds the S3 key, SHA-256 hash, transaction hash, gas used and anchoring time.
- Provisioned capacity totals 20 write and 20 read units. Dashboard tiles read only the latest item per device, and charts read the last 60 readings of the selected machine only.

Blockchain layer

- The `ServiceReportRegistry` contract has three parts:
  - `storeHash(reportId, fileHash)`: owner only, and an ID can be stored once, so a stored hash cannot be overwritten.
  - `verify(reportId, fileHash)`: a read-only call that returns true or false.
  - An event emitted on every store.
- The report ID is the Keccak-256 hash of the fault ID. The file hash is the SHA-256 of the PDF. Only these 32-byte values go on-chain, never the file.
- One dedicated test wallet owns the contract and signs all transactions from the dashboard. Its key is read from SSM and never written to code or logs.

Dashboard

- Streamlit runs on EC2 as a systemd service that restarts automatically. It uses the EC2 role, so no AWS keys are stored on the server.
- **Live Monitor:** 8 status tiles grouped by machine, plus a line chart refreshed every 5 seconds.
- **Faults:** list of faults with status.
- **Service Desk:** password protected. It uploads the PDF to S3, hashes it, anchors the hash, saves the report record, and sets the fault to ANCHORED.
- **Verify:** open to everyone. It compares an uploaded PDF with the stored hash and links to the transaction on Etherscan.

Security

- Per-device certificates and a topic-restricted policy secure ingestion.
- IAM roles give each component only the tables, bucket and parameters it needs.
- The S3 bucket is private and encrypted.
- The EC2 security group opens only the dashboard port, and admin access uses Session Manager, not SSH.
- The wallet holds test ETH only.

Stability and free-tier fit

- Sepolia keeps its data permanently, unlike a local test node that resets.
- Duplicate-alert protection stops repeated emails from filling the email allowance.
- Limits used:
  - Lambda allows 1 million requests per month, SNS allows 1,000 email deliveries per month, and DynamoDB allows 25 GB with 25 provisioned read and write units. Our usage is far below all three.
  - IoT Core's free allowance is 250,000 messages per month for 12 months. Our 8 devices send 2,880 messages per hour, so the simulator runs only during testing and demos.
  - Credits are applied automatically to usage beyond the always-free limits and to services without a free tier, which covers EC2 and S3 on new accounts. Stop the EC2 instance when it is idle.
  - Alchemy's free plan allows 30 million compute units per month, far more than the few calls we make. Stay on the Free plan.
- Check your AWS account type and creation date before relying on these limits.

Implementation Plan (5 Phases)

Standing conventions

- All AWS resources are created in **ap-south-1 (Mumbai)** with the prefix `plant-`.
- One Git repository (GitHub) with folders `simulator/`, `lambda/`, `contracts/`, `dashboard/` and `docs/`.
- No key, certificate or password is committed. Certificates stay in a git-ignored `certs/` folder.
- Library versions are pinned in `requirements.txt` and `package-lock.json` at build time.

Phase 1: Cloud Foundation and Guardrails (about 1 day)

Objective: Create the AWS resources that every later phase depends on, and protect the free-tier budget.

- **Account security:**
  - Enable MFA on the root user.
  - Create an IAM admin user for daily work.
  - Install and configure AWS CLI v2.
- **Cost guardrails:**
  - Enable AWS Free Tier usage alerts, which send an email when usage passes 85 percent of a free limit.
  - Create a monthly budget with an email alert.
- **DynamoDB tables (provisioned mode):**
  - `plant-readings`: partition key `device_id` (string), sort key `ts` (number). 10 write and 10 read units. TTL on the attribute `expire_at`.
  - `plant-faults`: partition key `device_id`, sort key `fault_id`. 5 write and 5 read units.
  - `plant-reports`: partition key `fault_id`. 5 write and 5 read units.
  - The total of 20 write and 20 read units stays inside the always-free allowance of 25 GB storage with 25 provisioned write and 25 provisioned read units.

- **S3 bucket** `plant-reports-<account-id>`: Block Public Access on, default SSE-S3 encryption.
- **SNS topic** `plant-fault-alerts` with one email subscription. Confirm it through the link in the confirmation email.
- **IAM roles (least privilege):**
  - `plant-iot-rule-role`: `dynamodb:PutItem` on `plant-readings` only.
  - `plant-lambda-role`: `dynamodb:PutItem` and `dynamodb:Query` on `plant-faults`, `sns:Publish` on the topic, and CloudWatch Logs.
  - `plant-ec2-role`: Query, GetItem, PutItem and UpdateItem on the three tables, `s3:GetObject` and `s3:PutObject` on `reports/*`, `ssm:GetParameter` on `/plant/*`, and the managed policy `AmazonSSMManagedInstanceCore`.

Exit criteria: Tables, bucket and topic exist. A test email is received. The usage alert and budget are active.

Phase 2: IoT Ingestion and Device Simulator (about 2 days)

Objective: Stream readings from 8 authenticated devices into DynamoDB.

- **Things and certificates:**
  - Create 8 IoT Things named `A1-MOTOR-TEMP`, `A2-MOTOR-CURR`, `A3-MOTOR-VIB`, `A4-PUMP-VIB`, `B1-MOTOR-TEMP`, `B2-MOTOR-CURR`, `B3-GBOX-VIB` and `B4-GBOX-TEMP`.
  - For each, create an active certificate and key pair, download both with Amazon Root CA 1, and attach the policy `plant-device-policy`.
- **Device policy:**
  - Allow `iot:Connect` only on the client ID equal to the Thing name.
  - Allow `iot:Publish` only on `plant/*/${iot:Connection.Thing.ThingName}/telemetry`.
  - Grant no subscribe permission.
- **Payload:** `{device_id, machine_id, part, sensor_type, value, unit, sent_at}` on topic `plant/{machine_id}/{device_id}/telemetry`.
- **Simulator (Python 3.12, AWS IoT Device SDK v2):**
  - One process creates 8 mutual-TLS connections using `mtls_from_path`, with client ID equal to the Thing name.
  - Each device publishes with QoS 1 every 10 seconds.
  - Values follow a bounded random walk around these baselines: temperature 55 to 75 °C, current 7 to 10 A, vibration 2 to 4.5 mm/s.
  - A console command `inject <device_id>` pushes that device above its limit, and `clear <device_id>` restores it.
  - The process stops automatically after a configurable duration (default 120 minutes) to protect the message quota.
- **Rule 1 (store everything):** `SELECT *, timestamp() AS ts, floor(timestamp()/1000) + 604800 AS expire_at FROM 'plant/+/+/telemetry'`. The action is DynamoDBv2 into `plant-readings`, and the error action writes to CloudWatch Logs.
- **Quota check:** 8 devices at one message per 10 seconds is 2,880 messages per hour. The free allowance is 250,000 messages per month for the first 12 months. Limiting runs to about 20 hours per month keeps usage near 57,600 messages.

Exit criteria:

- All 8 devices write rows for 10 minutes with no missing intervals.

- A test with the A1 certificate publishing to B1's topic is rejected.

Phase 3: Fault Detection and Alerting (about 1 to 2 days)

Objective: Detect limit violations automatically and send one alert per fault.

- **Rule 2:**
- `SELECT *, timestamp() AS ts FROM 'plant/+/+/telemetry'WHERE (sensor_type = 'temperature' AND value > 90)  OR (sensor_type = 'vibration' AND value > 7)  OR (sensor_type = 'current' AND value > 12)`

  The action invokes the Lambda function. Add the resource permission that lets `iot.amazonaws.com` invoke it from this rule.

- **Lambda `plant-fault-handler`** (Python 3.12, 128 MB, 10 second timeout, environment variables for table name, topic ARN and limits):
  - Build a `fault_id` such as `F-<epoch_ms>`.
  - Run one DynamoDB transaction that writes the fault item (status `OPEN`) and an `ACTIVE` marker item for that device with the condition `attribute_not_exists`.
  - If the condition fails, an open fault already exists, so exit without sending an alert.
  - If it succeeds, publish one SNS email containing device, part, value, limit, time and `fault_id`.
- **Cold start capture:** keep the CloudWatch REPORT lines, which show "Init Duration" for the first invocation.
- **Free-tier fit:** Lambda includes 1 million requests per month and SNS includes 1,000 email deliveries per month. Faults are rare, so usage is far below both.

Exit criteria:

- `inject A1` creates exactly one fault row and one email.
- Further high readings from A1 create no new alerts.
- Each of the other 7 devices also raises an alert when injected.

Phase 4: Blockchain Verification Layer (about 2 days)

Objective: Anchor and verify service-report hashes on a persistent public test network.

- **Contract `ServiceReportRegistry` (Solidity 0.8.x):**
  - `storeHash(bytes32 reportId, bytes32 fileHash)`: owner only, and rejects an ID that already exists.
  - `verify(bytes32 reportId, bytes32 fileHash) returns (bool)`: a read-only call.
  - Emit an event on every store. The file itself is never stored on-chain.
- **Hardhat project (Node.js 22 LTS):** tests cover a successful store, duplicate rejection, rejection of a non-owner, verify returning true for a matching hash and false for a changed hash, and a gas report.
- **Wallet:** generate one dedicated test account with `eth_account`. It holds only test ETH.
- **Test ETH:** use the pk910 Sepolia PoW faucet. A faucet comparison dated October 2026 says it pays out for mining work instead of requiring a mainnet balance. Request about 0.05 ETH one day early.

- **RPC:** create an Alchemy account and stay on its Free plan, which gives 30 million compute units per month with a limit of 500 compute units per second. Do not enable Pay As You Go.
- **Deploy** to Sepolia (chain ID 11155111) with Hardhat, and view the transaction on Etherscan Sepolia.
- **Store configuration** in SSM Parameter Store (standard tier):
  - `/plant/ledger/rpc_url`
  - `/plant/ledger/contract_address`
  - `/plant/ledger/private_key` as a SecureString
  - `/plant/dashboard/service_password` as a SecureString
- **Integration script:**
  - Generate a sample service-report PDF with `reportlab`.
  - Compute its SHA-256 hash with `hashlib`.
  - Call `storeHash` using the report ID `keccak256(fault_id)`.
  - Call `verify` with the original file and with a file with one changed byte.

Exit criteria: The original file returns MATCH, the altered file returns MISMATCH, and `gasUsed` and the transaction hash are recorded.

Phase 5: Dashboard, Integration and Evaluation (about 3 to 4 days)

Objective: Deploy the full application, run the experiments, and prepare the submission.

- **Streamlit app (Python 3.12), four pages:**
  - **Live Monitor:** 8 tiles grouped by machine and coloured by status. Tiles use a query with `Limit 1`. A line chart shows the last 60 readings of the selected machine only. Refresh uses `st.fragment(run_every="5s")`.
  - **Faults:** all faults with status OPEN or ANCHORED.
  - **Service Desk (password protected):** pick an open fault, upload the PDF, store it in S3 under `reports/{fault_id}.pdf`, hash it, send the signed `storeHash` transaction with web3.py, write the `plant-reports` row, and set the fault to ANCHORED. Delete the device's `ACTIVE` marker so a new fault can be raised.
  - **Verify:** upload a PDF, recompute the hash, call `verify`, and show MATCH or MISMATCH with an Etherscan link.
- **EC2 deployment:**
  - Launch a t3.micro with Ubuntu Server 24.04 LTS and an 8 GB volume, attach `plant-ec2-role`, and add a 1 GB swap file.
  - Open inbound TCP 8501 only, with no SSH. Administer through Session Manager.
  - Clone the repository, create a virtual environment, and install the requirements.
  - Run Streamlit as a systemd service with `Restart=always`.
- **Evaluation experiments:**
  - Alert delay: device `sent_at` to the fault `created_at`, repeated for each of the 8 devices.
  - Cold start: the first fault versus later faults, from the CloudWatch Init Duration.
  - Email delay: timed manually.
  - Gas: deployment and each `storeHash`, from the receipt.
  - Tamper test: matching versus altered file.
  - Security test: the rejected cross-device publish from Phase 2.
- **Comparison:** set the results beside the reference figures in the reviewed papers, namely the 2.0 s response in [17], the cold start delays in [5] and the gas figures in [15]. The comparison is a reference point only, since the systems differ.
- **Handover:**

    - Write the README, the architecture diagram, the measurement results, and a demo script of about 5 minutes.
    - Stop the EC2 instance after evaluation, and back up the repository and certificates.
    - The Free plan has a time limit: it ends after 6 months or when credits run out, whichever comes first. Check your account's creation date and finish before it expires.

Exit criteria: The full flow runs end to end from `inject` to alert, report upload, anchoring and verification, and all measurements are recorded.

Final Tech Stack

Cloud platform: AWS, region ap-south-1.

Device layer

- Simulator: Python 3.12 with AWS IoT Device SDK for Python v2 (`awsiotsdk`).
- Protocol: MQTT over TLS with X.509 client certificates.
- Ingestion and routing: AWS IoT Core (Things, policies, rules engine).

Cloud compute and data

- Fault handler: AWS Lambda (Python 3.12).
- Database: Amazon DynamoDB, provisioned mode, with TTL.
- Alerts: Amazon SNS email.
- File storage: Amazon S3, private bucket with SSE-S3.
- Hosting: Amazon EC2 t3.micro, Ubuntu Server 24.04 LTS, systemd.
- Secrets: AWS Systems Manager Parameter Store (standard tier, SecureString).
- Access control: AWS IAM roles with least privilege.
- Logging and monitoring: Amazon CloudWatch Logs.
- Cost control: AWS Free Tier usage alerts and AWS Budgets.
- Admin access: AWS Systems Manager Session Manager.

Application layer

- Dashboard: Streamlit with its built-in charts and `st.fragment` refresh.
- AWS access from Python: boto3.
- Hashing: Python standard library `hashlib` (SHA-256).
- Test PDFs: `reportlab`.

Blockchain layer

- Smart contract language: Solidity 0.8.x.
- Development and testing: Hardhat on Node.js 22 LTS.
- Network: Ethereum Sepolia testnet.
- Chain access from Python: web3.py, with signing through `eth_account`.
- RPC provider: Alchemy Free plan.
- Test ETH: pk910 Sepolia PoW faucet.
- Block explorer: Etherscan Sepolia.

Development tools

- Version control: Git and GitHub.
- Provisioning: AWS CLI v2 and the AWS Management Console.

Cost basis

- Lambda, SNS and DynamoDB use their always-free allowances.
- IoT Core uses the 12-month free allowance.
- On new accounts, EC2 and S3 are covered by Free Tier credits, because credits are applied automatically to usage beyond the always-free limits and to services with no free tier. Stop the EC2 instance when it is idle.
- Free-plan limits for Alchemy and the faucet can change, so check both when you set them up. I did not verify the free-plan limit for SSM Parameter Store.